DevOps

Tehnička praksa uvođenja principa toka
Automatizacija i omogućavanje niskorizičnih isporuka koda

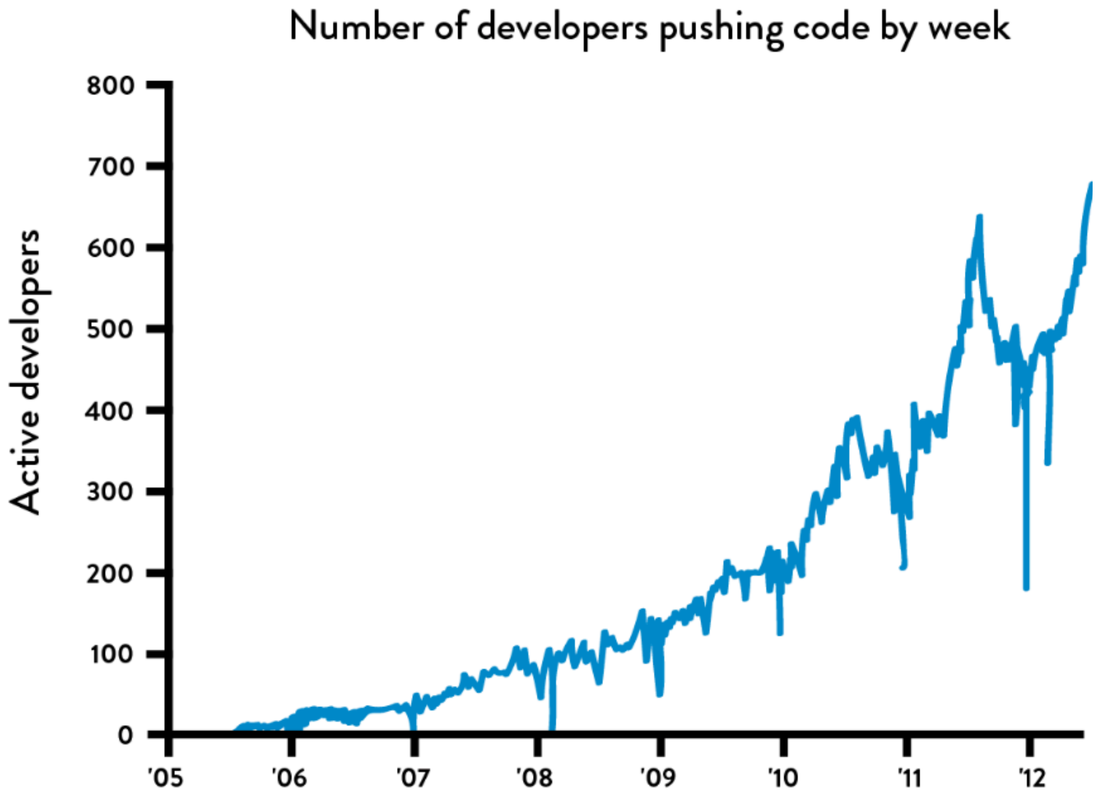
Tehničke prakse uvođenja principe toka

- Kontinualna isporuka (*Continuous Delivery*) podrazumeva
  - uspostavljanje *deployment pipeline-a*
  - omogućavanje brzog i pouzdanog automatizovanog testiranja
  - omogućavanje i praktikovanje kontinualne integracije
  - **automatizaciju i upravljanje razvojem koji omogućava isporuku uz male rizike (*low risk deployment*)**

Zašto je potrebna automatizacija procesa isporuke

- neautomatizovana isporuka koda zahteva da se većina tima u određenom delu radnog vremena "prebaci" na poslove *depolyment-a* i *releasovanja,* a to često može da bude i stresnije od rešavanja problema pod vremenskim ograničenjima

- iskustvo Facebook-a pokazuje da je automatizacija omogućila da se tokom dana poveća broj deployment-a, čak i u situaciji kada veliki broj programera dnevno komituje veliki broj izmena



Zašto je potrebna automatizacija procesa isporuke

- Ispostavilo se da je za veći broj funkcionalnih izmena neophodno veći broj deploymen-a, jer se pokazuje da postoji limit broja izmena koje uspeju da se uključe u jedan deployment.

- Ali kada se koristi kontinualna integracija i kada je sav kod već proveren onda je moguće obezbediti i automatizaciju samog procesa deploymenta, što čini sam postupak predvidljivim i niskorizičnim.

- Ovo značajno podiže produktivnost, jer proces manuelnog depolymenta je vremenski zahtevan, sklon greškama, i vrlo često je podrazumevao proces primopredaje između razvojnog i operativnog tima.

- Automatizacija procesa deployment-a obezbeđuje mogućnost promocije proverenog koda u produkciono okruženje (na zahtev ili automatizovano - odmah posle uspešnog builda i uspešno prođenih svih testova).

Automatizacija procesa isporuke (deployment-a)

- Neophodno je dokumentovati eventualno postojeći proces isporuke, a zatim sageldati koje je sve korake moguće automatizovati:
  - pakovanje koda
  - kreiranje unapred konfigurisanih slika virutuelnih mašina ili kontejnera
  - automatizacija deploymenta i konfiguracije *middlewear-a*
  - kopiranje paketa (ili fajlova) na produkciono okruženje
  - restartovanje servera, aplikacija ili servisa
  - generisanje konfiguracionih fajlova iz postojećih *template*-a
  - pokretanje *smoke* testova kako bi se uverili da sistem posle instalacije radi i da je korektno konfigurisan
  - izvršavanje test procedura
  - skripte i automatizacija neophodnih izmena u bazama podataka

Automatizacija procesa isporuke (deployment-a)

- razvojni tim treba da je uključen u automatizaciji i optimizaciji procesa isporuke, kako bi stekao iskustvo i postali svesni zavisnosti - ovo sikustvo nakandno im pomaže da se izbegnu situacije da male izmene u aplikaciji zahtevaju ponovni *deployment* ili promenu okruženja

Automatizacija procesa isporuke (deployment-a)

- Dosta alata koji nam omogućavaju kontinualnu integraciju podržava i ekstendovanje *pipeline-a* kako bi podržao i isporuku na produkciona ookruženja (tipično nakon acceptance testova)
  - Jenkins Build Pipeline plugin
  - ThoughtWorks Go.cd and Snap CI
  - Microsoft Visual Studio Team Services,
  - Pivotal Concourse

Zahtevi koji se postavljaju pred alate za automatizovanu isporuku (deployment)

- Istovetna isporuka na sva okruženja
  - korišćenje istog mehanizma isporuke za različita orkuženja (development, test, production) smanjuje mogućnost problema na produkciji
- Smoke testiranje isporučenog koda
- Obezbediti da se koriste konzistentna okruženja
  - Kada je uspostavljena istovetnost bilda za sva okruženja, neophodno je obezbediti da ta okruženja ostanu međusobno sinhronizvoana ako dolazi do nekih promena

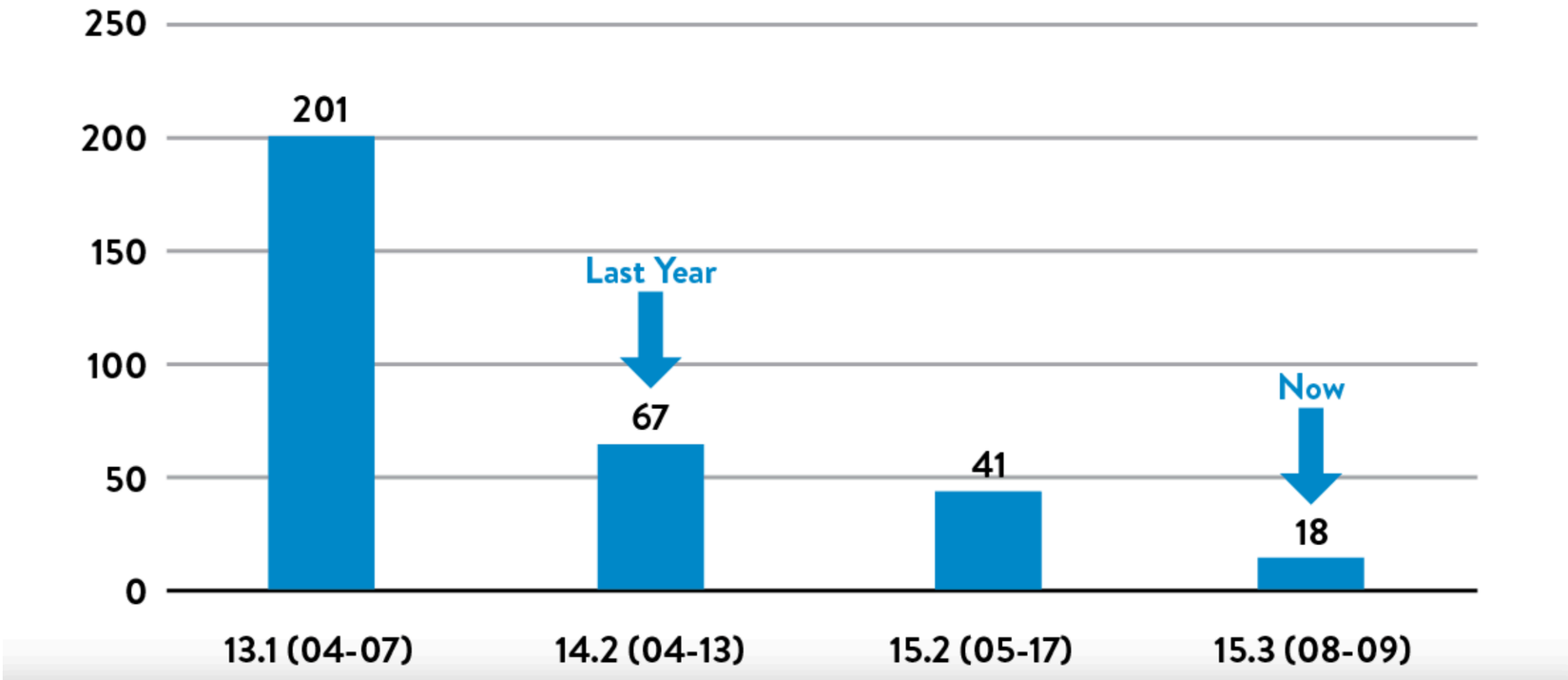
Primer uticaja automatizacije lansiranja nove verzije (release-a)

- Case Study SCG international
  - velika firma za izdavanje računa, imali su cikluse od dva releasa godišnje, i dok se razvojni tim navikao na korišćenje kontinualne integracije i deploymenta u development i test okruženja na dnevnoj bazi - operativni timovi su imali priliku da “testiraju” isporuku samo dva puta i to na visokorizičnom produkcionom okruženju, a sama test i razvojna okruženja nisu u potpunosti odgovarala produkcionom (firewalli, load balanceri…)
  - Operativni tim je integrisan u tim koji se bavio i dnevnim deploymentima na development i test okruženja, i na produkciona okruženja.
  - Ovo ih je motivisalo da automatizuju korake deploymenta, i kako su sada imali na stotine izvršenih depolymenta pre nego što treba isto da ponove u produkciji, automatizacija je drastično sputila broj grešaka, a broj produkcionih isporuka je dupliran.

Primer uticaja automatizacije lansiranja nove verzije (release-a)

Release Impact Improvements
Incidents by Release

Release	Incidents	Impact	Improvement
2013-Apr	201	455	0% (1x)
2014-Apr	67	153	66% (3x)
2015-May	41	97	79% (5x)
2015-Aug	18	45	90% (10x)



Omogućavanje samostalnog pokretanja isporuke (deploymenta) po potrebi

- Uspostavljanje čvrste kontrole i separacije nadležnosti između razvojnih i operativnih timova dovelo je do toga da su programeri “odsečeni” od mogućnosti da brzo isprave neke greške i isporuče izmene u produkciju.

- Postoje dobri razlozi da operativni timovi nadizru proces isporuke, ali…

- Statistički podaci pokazuju da nema velike razlike u uspešnosti isporuke koda u organizacijama gde to striktno obavlja operativni tim i u onima gde je to dozvoljeno i razvojnom timu

  - ukoliko je uspostavljena saradnja i zajednički cilj razvojnog i operativnog tima, i ukoliko je obezbeđena transparentnost, odgovornost za ishod isporuke koda, svejedno je ko će je obaviti

Omogućavanje samostalnog pokretanja isporuke (deploymenta) po potrebi

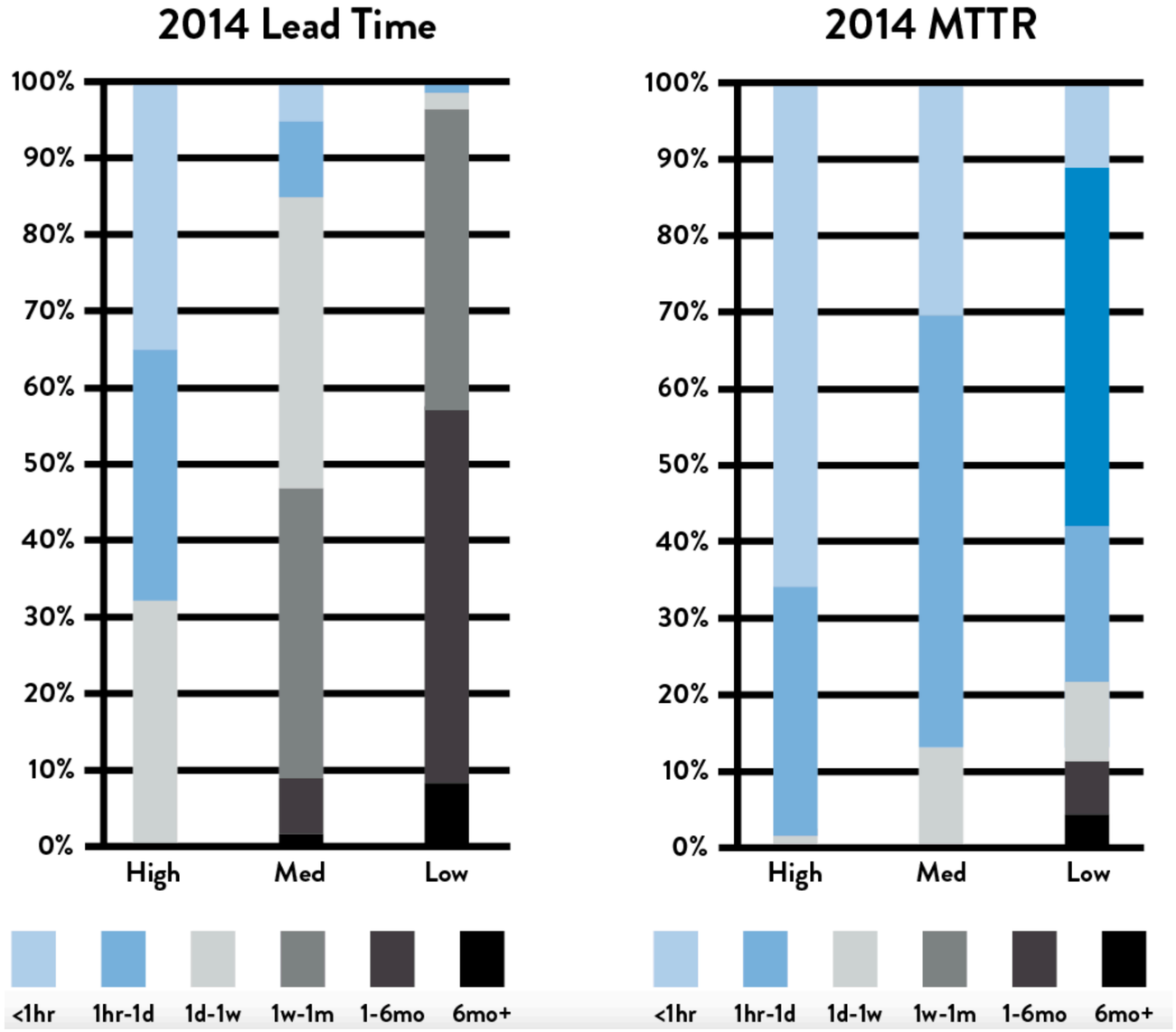
- Kako bi se postigli najbolji rezultati proces promocije koda na produkciju treba da mogu da obave i članovi razvojnog i operativnog tima, idealno bez manuelnih koraka. Da bi ovo bilo moguće i pouzdano neophodno je da:
  - *deployment pipeline* kreira pakete koda za distribuciju samo na osnovu koda koji je na sistemu za verzioniranje i da taj kod ispunjava uslov da je napravljen tako da je isporučiv na bilo koje od poznatih okruženja
  - svi treba da mogu da pokrenu određene ili sve automatizovane testove
  - svi članovi tima bi trebali da imaju mogućnost deploymenta na sva okruženja na koja imaju pravo pristupa

Integrisanje isporuke koda u pipeline

- kada su koraci isporuke koda automatizovani moguće ih je integrisati u *deployment pipeline.* Neophodno je:
  - osigurati da su paketi koji se kreiraju tokom kontinualne integracije u takvom stanju da su isporučivi na produkciju
  - omogućiti brz uvid u stanje okruženja na produkciji
  - obezbediti “dugme” kojim se na zahtev može pokrenuti isporuka na produkciono okruženje
  - automatski logovati sve akcije (komande) koje su izvršavane tokom isporuke koda na produkciju, kada, na kojim mašinama, ko ih je autorizovao
  - pokrenuti smoke testove
  - obezbediti brzu povratnu informaciju o statusu posle instalacije

Integrisanje isporuke koda u pipeline

- Automatizovana integrisana isporuka koda dovodi do drastičnog smanjenja vremena isporuke klijentu i vemena koj ej epotrebno da se obavi opopravak sistema



Razdvajanje isporuke i lansiranja koda

- Često je lansiranje nove verzije uslovljeno poslovnim i marketinsškim odlukama
    - uobičajen pristup - kompletan kod se prebacuje na produkciju prethodno veče, ujutro se objavljuje nova verzija itd. Ali onda mogu da se dese i neočekivani problemi u produkciji pa se radi “vatrogasni” roll-back, ili fix-forward.
- Česte isporuke omogućuju da se obaveprelazi na nove verzije bez većih trzavica

Razdvajanje isporuke i lansiranja koda

- Striktno razdvajanje isporuke koda (deployment) i lansiranja verzije za klijente pomaže da se prevaziđu ovi problemi
  - Isporuka koda (deployment) predstavlja instalaciju specifične verzije softvera na određeno okruženje. Sama isporuka ne mora biti istovremeno povezana i sa objavljivanjem određenih funkcionalnosti krajnjim korisnicima.
  - lansiranje verzije koda se dešava kada se određene funkcionalnosti ili grupa njih učine dostupne krajnjim klijentima. Kod i sama arhitektura sistema treba da su takvi da omogućavaju da se ovo lansiranje novih funkcionalnosti obavlja bez potrebe za dodatnim izmenama u kodu.
- Ukoliko su proces instalacije i lansiranja određenih funkcionalnosti odvojeni, onda objavljivanje novih funkcionalnosti više nije tehnički problem (jer sve je većć ranije instalirano i testirano), već pitanje poslovne odluke kada omogućiti pristup određenim funkcionalnostima

Razdvajanje isporuke i lansiranja koda

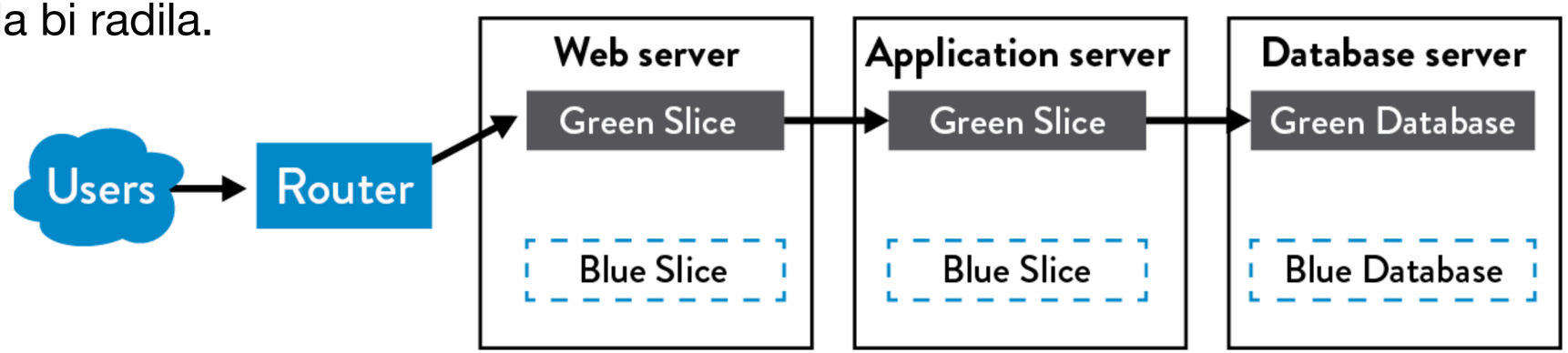
- Postoje dve kategorije šablona za lansiranje novih verzija koda:

  - lansiranje (release) na osnovu okruženja - kada imamo dva ili više okruženja na koje se instalira kod, ali samo jedno okruženje prima živi saobraćaj od krajnjih korisnika. Novi kod se instalira na ona okruženja koja ne primaju živi saobraćaj i tamo se testira. U tom slučaju lansiranje nove verzije se izvodi preusmeravanjem saobraćaja na drugo okruženje.
    - *blue-green deployment, canary releases, cluster immune systems*

  - lansiranje (release) na osnovu aplikacija - aplikacija se piše tako da je moguće napraviti izmene i sakriti ili učiniti dostupnim određene funkcionalnosti izmenom konfiguracionih fajlova. Ovo omogućava i tehniku tzv. *dark launching* - kada se sve funkcionalnosti instaliramo uz producioni softver i testiramo pod realnim opterećenjem, ali su te funkcionalnosti skrivene od krajnjih korisnika, dok se ne uverimo da je sve OK

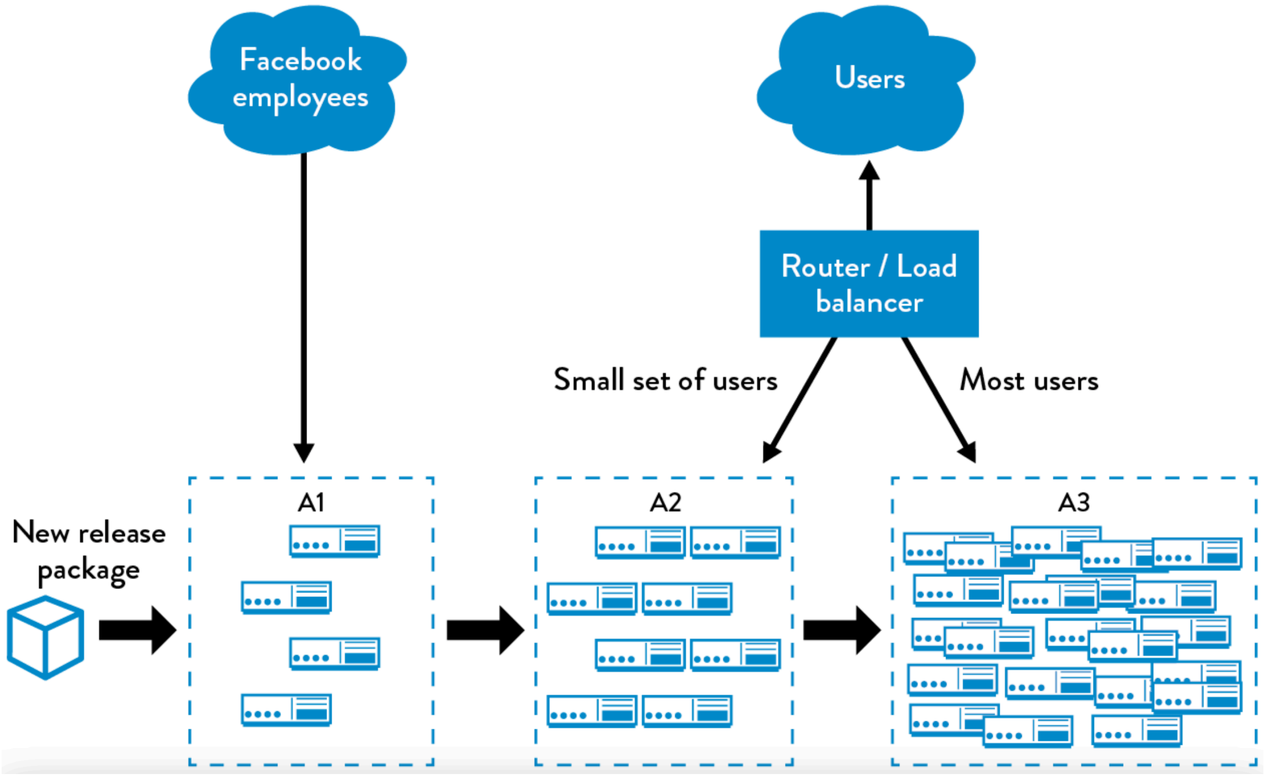
Blue green deployment pattern

- Postoje dva produkciona okruženja (tzv. plavo i zeleno) u svakom momentu, ali samo jedno od njih prima saobraćaj od korisnika (za početak npr. plava). Nove verzije se instaliraju na pasivno okruženje (zeleno), tamo se sve testira, i kada smo uvereni da je sve ok, saobraćaj korisnika se preusmeri na to okruženje.
- Ukoliko je potrebno izvršiti roll-back on se vrši jednostavno - saobraćaj se vrati na prvobitno aktivno okruženje (plavo).
- Problem baza podataka:
  - dve baze - po jedna za svaku instancu koje se prilikom lansiranja nove verzije usinhronizuju kako bi novi sistem imao sve podatke sa do tada aktivnog. Problem se javlja ako se mora uraditi rolll-back, jer se moraju migrirati i sve transakcije koje su u međuvremenu nastale
  - odvajanje verzije baze od promena u aplikaciji - aplikacije se pišu tako da njihovo lansiranje ne zavisi od verzije baze: 1. dozvoljavaju se samo aditivne promene u bazi, 2. aplikacija ne pretpostavlja tačno određenu verziju baze da bi radila.



Canary release pattern

- Automatizuje proces lansiranja nove verzije tako što se novi kod sukcesivno instalira na sve složenija okruženja, ali samo pod uslovom da je na prethodnom okruženju sve uspešno testirano.
- Ukoliko se na određenom okruženju otkrije greška, radi se roll-back i novi kod se ne promoviše dalje.
- Primer:
  - A1 - grupa servera za lokalnu upotrebu
  - A2 - grupa servera koju može da koristi limitirani broj krajnjih korisnika nakon što je potvrđen acceptance
  - A3 - puno produkciono okruženje



Cluster immune system pattern

- Modifikacija *canary pattern-a* tako što se povezuje monitroing sistem na produkciji sa automatizovanim *release* procesom, tako da kada monitoring sistem otkrije da performanse na produkcionom sistemu nakon pokretanja nove verzije padaju ispod određenih unapred postavljenih parametara, automatski trigeruje roll-back.

- Ovo dodatno smanjuje vreme reakcije i oporavka sistema

Obrasci lansiranja nove verzije bazirani na aplikacijama

- **Implementiranje mogućnosti uključivanja/isključivanja funkcionalanosti (feature toggles).**
  - Obično se implementira tako što se delovi interfejsa vidljivi krajnjem korisniku i funkcionalnosti kojima se pristupa preko tog interfejsa “umotavaju” u uslovne naredbe
  - Omogućava lak povratak na verziju bez date funkcionalnosti (rolll-back), smanjenje funkcionalnosti uz očuvanje aktivnosti aplikacije (gracefull degradation of performance), povećanje otpornosti sistema baziranog na servisnim konceptima (moguće je implementirati funkcionalnost koja se oslanja na servis, čak i kada on ne radi u potpunosti i uključiti je kada sve proradi, ili isključiti ako servis ima problema)

Obrasci lansiranja nove verzije bazirani na aplikacijama

- **Korišćenje lansiranja aplikacije “u mraku” - *dark launches***
  - Ovaj konept je u suštini posledica mogućnosti da se funkcionalnosti uključe isključe. Ideja je da se aplikacija sa svim funkcionalnostima instalira na produkciono okruženje, ali da se određene funkcionalnosti ne uključuju za krajnjeg korisnika, ali se za to vreme testiraju u potpuno relanom okruženju. Krajnji korisnici su nesvesni da nova funkcionalnost postoji sve do momenta kada smo zadovoljni testiranjem i uključimo je i za njih.